Horrible Code Activity

Calvin Zheng, Nikhil Vundela, Sri Bhargav Reddy Pedhiti

Calculator Program: Good Code vs. Bad Code

This activity demonstrates coding best practices by comparing two versions of a simple calculator program. Both versions allow the user to choose an operation and enter two numbers, but the structure and quality of the code are different. The bad version intentionally violates recommended programming practices, while the good version fixes those issues.

Program Description

The program is a basic calculator that supports addition, subtraction, multiplication, and division. The user selects an operator, enters two numbers, and the program displays the result. The program uses multiple functions, satisfying the assignment requirement for at least three functions.

Functions Used

Version	Functions	Purpose
Bad Code	x(), y(), z(), w()	Each function asks for input, performs a calculation, and prints output.
Good Code	get_numbers(), calculate(), display_result(), main()	Each function has a clear role: input, calculation, output, or program execution.

Selected Principles Demonstrated or Violated

1. KISS

The bad code violates KISS because it repeats print statements and spreads simple logic across unclear functions. The good code follows KISS by keeping the calculator simple and avoiding unnecessary repetition.

2. DRY Code

The bad code violates DRY because the same input prompts and completion messages appear in several places. The good code improves this by reusing get_numbers() for input and printing the completion message once inside display_result().

3. Single Responsibility

The bad code violates single responsibility because each operation function handles input, calculation, and output at the same time. The good code fixes this by separating responsibilities into different functions.

4. Separation of Concerns

The bad code mixes user input, math logic, and output formatting together. The good code separates these concerns so that changing one part of the program does not require rewriting the whole program.

5. Clean Code

The bad code uses unclear names like x, y, z, w, a, b, c, and d. These names do not explain what the code does. The good code uses descriptive names like get_numbers, calculate, display_result, first_number, and second_number.

6. Document Your Code

The good code includes comments and docstrings explaining the purpose of each major function. The bad code has comments explaining violations, but the function and variable names still make the code harder to

understand.

How the Bad Version Violates Best Practices

- Unclear function names make the program harder to read.

- The same input logic is repeated in every operation function.

- The same completion message is printed multiple times.

- Each function does too many jobs at once.

- There is no division-by-zero protection in the division function.

- The main program logic runs directly instead of being organized inside a main() function.

How the Good Version Fixes the Issues

- The code uses clear function names that explain their purpose.

- Input is handled once in get_numbers().

- Math logic is handled in calculate().

- Output is handled in display_result().

- main() controls the overall program flow.

- The division operation checks for division by zero before dividing.

- The entry point guard prevents the program from running automatically if imported into another file.

Conclusion

The two calculator programs show how coding best practices affect readability, maintainability, and reliability. The bad version technically works for basic calculations, but it is harder to maintain because it

repeats code, uses unclear names, and mixes responsibilities. The good version is cleaner because each function has one clear purpose, repeated code is reduced, and the program is easier to understand and update.

Screenshots of commits: